

4장 추상 구문

최광훈

전남대학교

목차

추상 구문 트리(AST)

방문자 디자인 패턴(Visitor Design Pattern)

[실습] MiniJava 추상 구문 트리 (방문자 패턴 사용)

추상 구문 트리(AST)

파싱과 의미 분석(타입 체킹)을 분리하기 위해 추상 구문 트리가 필요

- 파서는 추상 구문 트리를 만들고,
- 후속 단계로 이 트리를 탐색하며 타입 체킹

추상 구문 트리 없이 파싱하면서 실행하는 방법도 가능하지만,

- 교재 4.1의 프로그램 4.2 예제
- 파싱 순서 그대로만 분석하게 되어 유연성이 떨어지고, 컴파일러 유지보수도 어려움

```

void Start() :
{ int i; }
{ i=Exp() <EOF> { System.out.println(i); }
}

int Exp() :
{ int a,i; }
{ a=Term()
  ( "+" i=Term() { a=a+i; }
  | "-" i=Term() { a=a-i; }
  ) *
  { return a; }
}

int Term() :
{ int a,i; }
{ a=Factor()
  ( "*" i=Factor() { a=a*i; }
  | "/" i=Factor() { a=a/i; }
  ) *
  { return a; }
}

int Factor() :
{ Token t; int i; }
{ t=<IDENTIFIER> { return lookup(t.image); }
| t=<INTEGER_LITERAL> { return Integer.parseInt(t.image); }
| "(" i=Exp() ")" { return i; }
}

```

추상 구문 트리(AST)

파스 트리 (Parse tree) 또는 구문 트리 (Concrete syntax tree)

- 파싱 중 사용된 각 문법 규칙의 내부 노드 존재
- 소스 프로그램의 문법 구조를 반영

한계

- 괄호, 세미콜론 등 구두점 토큰이 많아 불필요하게 복잡
- 문법 변환 과정(예: **left factoring**)에서 추가된 비단말/생산 규칙 때문에
의미 분석 단계까지 문법 세부사항이 드러남

추상 구문 트리(AST)

추상 구문 트리와 문법의 관계

- **AST**용 문법은 파싱을 위한 문법이 아님
- 실제 파서는 구문 문법에 따라 입력(프로그램 텍스트)을 읽고
- 그 결과를 **AST** 형태로 구성

따라서,

- **AST** 문법이 다소 모호해도 괜찮음
- 이미 파싱이 끝난 뒤이므로 후속 컴파일 단계는 문법 모호성을 다시 고민할 필요가 없음

추상 구문 트리(AST)

산술식 언어의 AST 문법

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow \text{id}$$

$$E \rightarrow \text{num}$$

GRAMMAR 4.3. Abstract syntax of expressions.

추상 구문 트리(AST)

추상 구문 트리 구현 방식

- 구현 언어 **Java**를 기준으로 앞의 산술식 언어 **AST** 문법을 구현하는 방법

1) 비단말(nonterminal) -> 추상 클래스

2) 각 생산 규칙(production) -> 하위 클래스

- 예) 추상 클래스 Exp (비단말 E)

- 예) 하위 클래스 PluseExp, MinusExp, TimesExp, DivideExp

($E \rightarrow E + E \mid E - E \mid E * E \mid E / E$)

- 예) 하위 클래스 Identifier, IntegerLiteral ($E \rightarrow Identifier \mid IntegerLiteral$)


```

public abstract class Exp {
    public abstract int eval();
}

public class PlusExp extends Exp {
    private Exp e1,e2;
    public PlusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()+e2.eval();
    }
}

public class MinusExp extends Exp {
    private Exp e1,e2;
    public MinusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()-e2.eval();
    }
}

public class TimesExp extends Exp {
    private Exp e1,e2;
    public TimesExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()*e2.eval();
    }
}

public class DivideExp extends Exp {
    private Exp e1,e2;
    public DivideExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()/e2.eval();
    }
}

```

```

public class Identifier extends Exp {
    private String f0;
    public Identifier(String n0) { f0 = n0; }
    public int eval() {
        return lookup(f0);
    }
}

public class IntegerLiteral extends Exp {
    private String f0;
    public IntegerLiteral(String n0) { f0 = n0; }
    public int eval() {
        return Integer.parseInt(f0);
    }
}

```

PROGRAM 4.5. Exp class for Program 4.4.

추상 구문 트리(AST)

추상 구문 트리 노드에 위치 저장

- 위치: 프로그램 텍스트 상에서의 줄(line), 열(column) 번호
- 이 노드가 원본 소스의 어느 위치에서 왔는지 기록
- 타입 오류, 의미 오류가 발생하면 해당 노드의 줄과 열 번호로 정확한 위치를 제시

AST 위치 정보

- 잎(leaf) 노드 : 어휘 분석기가 반환한 토큰의 시작과 끝
- 내부 노드 : 자식 노드들의 위치로부터 계산 (첫째 자식의 시작 ~ 마지막 자식의 끝)

프로그램 4.4: 구문 트리를 생성하는 파서 액션 코드가 작성된 JavaCC 문법

구문 문법

```
Start -> Exp
Exp -> Term ( + Term
      | - Term ) *
Term -> Factor ( * Factor
      | / Factor ) *
Factor -> identifier
      | interger_literal
      | ( Exp )
```

```
Exp Start() :
{
    Exp e;
    { e=Exp() { return e; }
    }
}

Exp Exp() :
{
    Exp e1,e2;
    { e1=Term()
      ( "+" e2=Term() { e1=new PlusExp(e1,e2); }
      | "-" e2=Term() { e1=new MinusExp(e1,e2); }
      ) *
      { return e1; }
    }
}

Exp Term() :
{
    Exp e1,e2;
    { e1=Factor()
      ( "*" e2=Factor() { e1=new TimesExp(e1,e2); }
      | "/" e2=Factor() { e1=new DivideExp(e1,e2); }
      ) *
      { return e1; }
    }
}

Exp Factor() :
{
    Token t; Exp e;
    { ( t=<IDENTIFIER> { return new Identifier(t.image); } |
      t=<INTEGER_LITERAL> { return new IntegerLiteral(t.image); } |
      "(" e=Exp() ")" { return e; } )
    }
}
```

AST 문법

```

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow id$$

$$E \rightarrow num$$

```

추상 구문 트리(AST)

추상 구문 트리 노드에 위치 저장

- 위치: 프로그램 텍스트 상에서의 줄(line), 열(column) 번호
- 이 노드가 원본 소스의 어느 위치에서 왔는지 기록
- 타입 오류, 의미 오류가 발생하면 해당 노드의 줄과 열 번호로 정확한 위치를 제시

AST 위치 정보

- 잎(leaf) 노드 : 어휘 분석기가 반환한 토큰의 시작과 끝
- 내부 노드 : 자식 노드들의 위치로부터 계산 (첫째 자식의 시작 ~ 마지막 자식의 끝)

목차

추상 구문 트리(AST)

방문자 디자인 패턴(Visitor Design Pattern)

[실습] MiniJava 추상 구문 트리 (방문자 패턴 사용)

방문자 패턴

디자인 패턴

- 객체지향 프로그래밍에서 재사용성을 높이기 위한 클래스 구조, 상속, 객체 조합 방법

방문자 패턴 (Visitor patterns)

- 객체의 클래스를 수정하지 않고도
- 그 객체 구조에 새로운 연산을 추가할 수 있는 방법을 제공

방문자 패턴

방문자 패턴을 사용 ...

- 사전에 정의된 클래스 집합
- 각 클래스는 **accept** 메소드를 갖춤

1차 시도: instanceof와 타입 cast

Java 예제 실행 : 정수 리스트 합 구하기

```
interface List {}
```

```
class Nil implements List {}
```

```
class Cons implements List {  
    int head;  
    List tail;  
}
```

```
List l;        // The List-object
```

```
int sum = 0;
```

```
boolean proceed = true;
```

```
while (proceed) {
```

```
    if (l instanceof Nil)
```

```
        proceed = false;
```

```
    else if (l instanceof Cons) {
```

```
        sum = sum + ((Cons) l).head;
```

```
        l = ((Cons) l).tail;
```

```
        // Notice the two type casts!
```

```
    }
```

```
}
```


1차 시도: instanceof와 타입 cast

Java 예제 실행 : 정수 리스트 합 구하기

```
interface List {}
```

```
class Nil implements List {}
```

```
class Cons implements List {  
    int head;  
    List tail;  
}
```

장점: sum 함수를 작성하더라도 Nil이나 Cons를 수정할 필요가 없음

단점: 타입 cast와 instanceof를 반복해서 지나치게 많이 사용

```
List l;          // The List-object  
int sum = 0;  
boolean proceed = true;  
while (proceed) {  
    if (l instanceof Nil)  
        proceed = false;  
    else if (l instanceof Cons) {  
        sum = sum + ((Cons) l).head;  
        l = ((Cons) l).tail;  
        // Notice the two type casts!  
    }  
}
```

2차 시도: sum 담당 함수 도입

Java 예제 실행 : 정수 리스트 합 구하기

```
class Nil implements List {
    public int sum() {
        return 0;
    }
}

class Cons implements List {
    int head;
    List tail;
    public int sum() {
        return head + tail.sum();
    }
}
```

장점: 타입 변환(**cast**)과 타입 검사 연산(**instanceof**)
이 사라져, 코드를 체계적으로 작성할 수 있음

단점: 새로운 기능(**prettyPrint**, **typeCheck**,
translateToIR, ...)이 추가될 때마다 그 연산을 위한
전용 메서드를 새로 작성해야 하며, 모든 클래스를
다시 컴파일해야 함.

3차 시도: 방문자 패턴

핵심 아이디어: 코드를 객체 구조와 **Visitor**로 분리(함수형 프로그래밍과 유사한 발상)

(1) 각 클래스(예: **Nil**, **Cons**)에 **accept** 메서드를 추가,

Visitor를 인자로 받음

(2) **Visitor**는 각 클래스마다 하나의 **visit** 메서드를 갖춤(메서드 오버로딩),

클래스 **C**에 대한 **visit** 메서드는 타입 **C**의 객체를 인자로 받음

3차 시도: 방문자 패턴

(1) 각 클래스(예: Nil, Cons)에 `accept` 메서드를 추가, `Visitor`를 인자로 받음

```
interface List {  
    void accept(Visitor v);  
}
```

```
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

```
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

3차 시도: 방문자 패턴

(2) Visitor는 각 클래스마다 하나의 visit 메서드를 갖춤(메서드 오버로딩),

클래스 C에 대한 visit 메서드는 타입 C의 객체를 인자로 받음

```
interface List {  
    void accept(Visitor v);  
}
```

```
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

```
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

3차 시도: 방문자 패턴

실행 흐름은 **Visitor**의 **visit** 메서드와 객체 구조의 **accept** 메서드가 번갈아 호출되면서 진행

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}  
  
SumVisitor sv = new SumVisitor();  
l.accept(sv);  
System.out.println(sv.sum);
```

메모: **visit** 메서드는 수행할 연산(+)과 하위 객체를 따라가는 방식(**x.tail**)을 모두 구현

3차 시도: 방문자 패턴

Visitor 패턴은 앞의 두 접근 방식의 장점을 결합

Visitor의 장점

- 재컴파일 없이 새로운 연산(메서드)을 추가 가능
- Visitor 사용의 전제 조건: 모든 클래스가 **accept** 메서드를 가져야 함

	타입 cast 를 자주 사용?	재컴파일 자주 필요?
instanceof , 타입 cast	Yes	No
전담 메소드	No	Yes
방문자 패턴	No	No

Visitor 패턴을 사용하는 도구: JJTree, JTB(Java Tree Builder)

방문자 패턴: Exp 클래스

```

public abstract class Exp {
    public abstract int accept(Visitor v);
}

public class PlusExp extends Exp {
    public Exp e1,e2;
    public PlusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}

public class MinusExp extends Exp {
    public Exp e1,e2;
    public MinusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}

public class TimesExp extends Exp {
    public Exp e1,e2;
    public TimesExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}

```

```

public class DivideExp extends Exp {
    public Exp e1,e2;
    public DivideExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}

public class Identifier extends Exp {
    public String f0;
    public Identifier(String n0) { f0 = n0; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}

public class IntegerLiteral extends Exp {
    public String f0;
    public IntegerLiteral(String n0) { f0 = n0; }
    public int accept() {
        return v.visit(this);
    }
}

```

PROGRAM 4.7. Syntax classes with accept methods.

방문자 패턴: 방문자 클래스 Interpreter

```
public interface Visitor {
    public int visit(PlusExp n);
    public int visit(MinusExp n);
    public int visit(TimesExp n);
    public int visit(DivideExp n);
    public int visit(Identifier n);
    public int visit(IntegerLiteral n);
}
```

```
public class Interpreter implements Visitor {
    public int visit(PlusExp n) {
        return n.e1.accept(this)+n.e2.accept(this);
    }
    public int visit(MinusExp n) {
        return n.e1.accept(this)-n.e2.accept(this);
    }
    public int visit(TimesExp n) {
        return n.e1.accept(this)*n.e2.accept(this);
    }
    public int visit(DivideExp n) {
        return n.e1.accept(this)/n.e2.accept(this);
    }
    public int visit(Identifier n) {
        return lookup(n.f0);
    }
    public int visit(IntegerLiteral n) {
        return Integer.parseInt(n.f0);
    }
}
```

방문자 패턴 (요약)

새로운 기능 추가가 쉽다: 새 방문자 클래스만 작성하면 됨

- 예: `prettyPrint`, `typeCheck`, `translate`, ...

관련 연산은 모으고, 무관한 연산은 분리할 수 있다.

새 클래스 추가는 어렵다.

- 연산은 자주 바뀌고, 데이터 구조(예: `Cons`, `Nil`)는 안정적일 때 적합하다.

상태 누적이 가능하다. (예: `SumVisitor.sum`)

캡슐화를 약화시킬 수 있다. (예: `x.head`, `x.tail`)

목차

추상 구문 트리(AST)

방문자 디자인 패턴(Visitor Design Pattern)

[실습] MiniJava 추상 구문 트리 (방문자 패턴 사용)

[실습] MiniJava AST 클래스와 방문자 클래스

3장 실습에서 작성한 JavaCC 파서에 의미 동작(semantic action)을 추가하여 MiniJava 언어의 추상 구문을 생성하시오.

구문 트리 클래스

- chap4/handcrafted/syntaxtree/{ClassDecl,MethodDecl,Exp,...}.java

방문자 클래스

- chap4/visitor/{Visitor,PrettyPrintVistor, ... }.java

MiniJava AST 클래스

Program(MainClass m, ClassDeclList cl)

MainClass(Identifier i1, Identifier i2, Statement s)

abstract class ClassDecl

ClassDeclSimple(Identifier i, VarDeclList vl, MethodDeclList ml)

ClassDeclExtends(Identifier i, Identifier j,
VarDeclList vl, MethodDeclList ml) *see Ch.14*

VarDecl(Type t, Identifier i)

MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,
StatementList sl, Exp e)

Formal(Type t, Identifier i)

abstract class Type

IntArrayType() **BooleanType**() **IntegerType**() **IdentifierType**(String s)

MiniJava AST 클래스

```
abstract class Statement
  Block(StatementList s1)
  If(Exp e, Statement s1, Statement s2)
  While(Exp e, Statement s)
  Print(Exp e)
  Assign(Identifier i, Exp e)
  ArrayAssign(Identifier i, Exp e1, Exp e2)
```

MiniJava AST 클래스

```
abstract class Exp
And(Exp e1, Exp e2)
LessThan(Exp e1, Exp e2)
Plus(Exp e1, Exp e2) Minus(Exp e1, Exp e2) Times(Exp e1, Exp e2)
ArrayLookup(Exp e1, Exp e2)
ArrayLength(Exp e)
Call(Exp e, Identifier i, ExpList el)
IntegerLiteral(int i)
True()
False()
IdentifierExp(String s)
This()
NewArray(Exp e)
NewObject(Identifier i)
Not(Exp e)

Identifier(String s)
```

MiniJava AST 클래스

list classes

ClassDeclList () ExpList () FormalList () MethodDeclList () StatementList () VarDeclList ()

MiniJava 방문자 클래스

```
public interface Visitor {  
    public void visit(Program n);  
    public void visit(MainClass n);  
    public void visit(ClassDeclSimple n);  
    public void visit(ClassDeclExtends n);  
    public void visit(VarDecl n);  
    public void visit(MethodDecl n);  
    public void visit(Formal n);  
    public void visit(IntArrayType n);  
    public void visit(BooleanType n);  
    public void visit(IntegerType n);  
    public void visit(IdentifierType n);  
    public void visit(Block n);  
    public void visit(If n);  
    public void visit(While n);  
    public void visit(Print n);  
    public void visit(Assign n);  
    public void visit(ArrayAssign n);
```

```
    public void visit(And n);  
    public void visit(LessThan n);  
    public void visit(Plus n);  
    public void visit(Minus n);  
    public void visit(Times n);  
    public void visit(ArrayLookup n);  
    public void visit(ArrayLength n);  
    public void visit(Call n);  
    public void visit(IntegerLiteral n);  
    public void visit(True n);  
    public void visit(False n);  
    public void visit(IdentifierExp n);  
    public void visit(This n);  
    public void visit(NewArray n);  
    public void visit(NewObject n);  
    public void visit(Not n);  
    public void visit(Identifier n);  
}
```

MiniJava AST 클래스

list classes

ClassDeclList () ExpList () FormalList () MethodDeclList () StatementList () VarDeclList ()